Tutorial 7

Core exercises

1. **Minimizing a function in 1D**
In this exercise we will implement some of the algorithms to find the maximum or minimum of functions in 1D. Consider the following function:

$$f(x) = x^4 + 10x^3 + 10(x-2)^2 \tag{1}$$

This function has 2 minima in the range $[-10, 10]$, one of which is a local minimum.

(a) Find its minima using the bracketing procedure and your choice between the Golden section and Brent's method. Bonus if you implement all three methods.

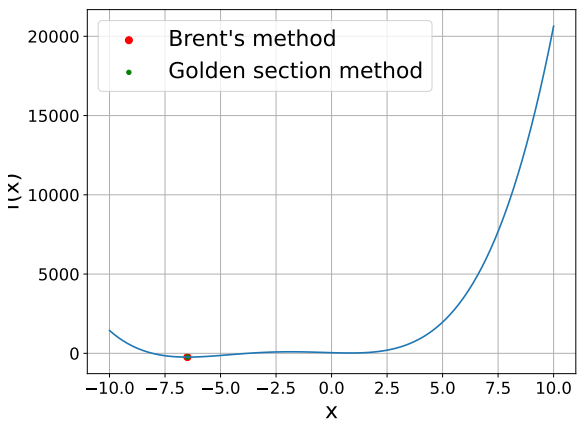
(b) Now consider a different function:

$$g(x) = \cos(x) + \frac{x^2}{20}. \tag{2}$$

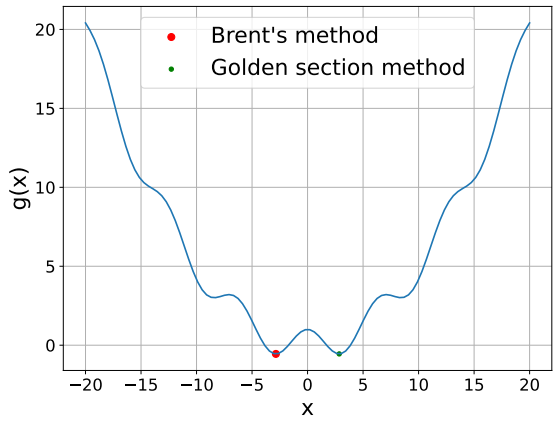
This function has several (local) minima and several (local) maxima in the range $(-20, 20)$, and global minima as well. Find the minima and maxima in this range using the methods developed for point (a).

Solution:

(a) See Lecture 7 slide 7 for implementing the bracketing, and slide 11 for inverse parabolic interpolation.
Check that the points are ordered correctly before tightening the bracket.
Beware that when you change the ordering of your a and b in the golden section search, the proposed c will be smaller than b, which means that the condition "if d was between b and c" has 2 options depending on $c \gtrless b$.
In Brent's method, w is the second-closest to the minimum point. At each step, its value is updated to b or d, depending on which one corresponds to a smaller value of the function. Another quantity, v, is the third-closest to the minimum. It is usually updated to w at the previous iteration, but sometimes v can be updated to d if the new point d does not approach the minima closer than b or w. The results are shown in the plot below.



(b) This function has multiple symmetrical local minima, and 2 global ones. The minimum found by your routine depends on the initial bracketing (and w in the golden section search).



2. **Minimizing a function in 2D**
In this exercise we will look at two different minimization techniques: the downhill simplex method and the quasi-Newton method. We will start from simple functions and then move to more challenging ones.

(a) We first consider the following function:

$$f(x, y) = -\exp\left(-x^2 - y^2\right). \tag{3}$$

Make a contour plot of this function.

(b) Use the downhill simplex method to find the minimum. Start from the point $(5, 5)$ and store the vertex coordinates of the triangles in an array/matrix.

(c) Plot the triangles in each time step and make a movie of how the triangles evolve as downhill simplex gets closer to the minimum.

(d) Make a plot of the distance to the minimum versus the number of steps. Bonus if you create a movie with a two-panel plot, one showing at each step the evolution of the triangles, the other presenting the corresponding progress of the distance to the minimum.

(e) Now use the quasi-Newton method to find the minimum of the function, store the different points in an array and make a movie of how the quasi-Newton method finds the minimum.

(f) Let's now consider a more complex function:

$$g(x, y) = \sqrt{(5 + 2\sqrt{3})x^2 - 4xy + (5 - 2\sqrt{3})y^2}. \tag{4}$$

Repeat the same five steps as done for the first function, using $(1.5, 2.5)$ as starting point.

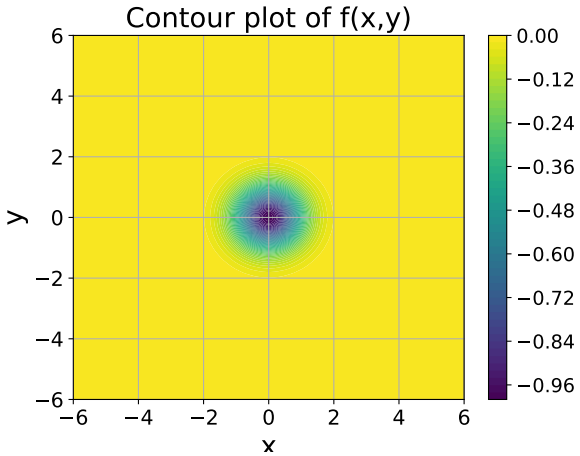
(g) Bonus: The following function was specifically designed to test the efficiency of optimization methods (such as finding minima):

$$h(x, y) = 100(y - x^2)^2 + (1 - x)^2. \tag{5}$$

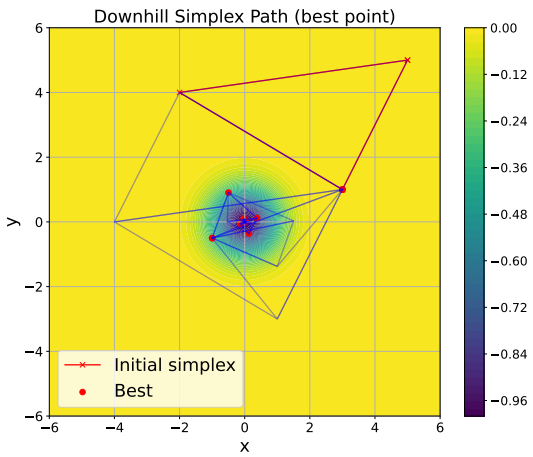
Start by plotting the contours of this function. Inspect the plot and explain why this function represents a good test-bed for function minimization algorithms. Repeat steps (a) though (e), using $(-0.5, 0.5)$ as starting point.

Solution:

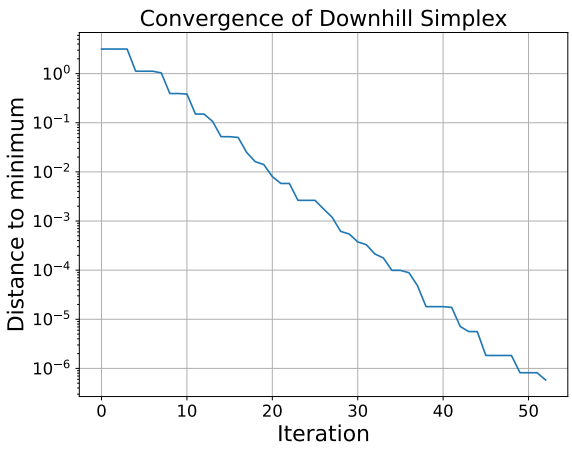
(a) The function, shown below, has a single sharply localised global minimum at (0,0).



(b) In downhill simplex, do not forget to recompute your centroid each iteration before proposing a new point. Another tip, to prevent stopping at the plateau where $f(\mathbf{x}_N) \approx f(\mathbf{x}_0)$, you can impose additional conditions for simplex size (e.g., maximal distance between $\mathbf{x}_N$ and $\mathbf{x}_0$).



(c)



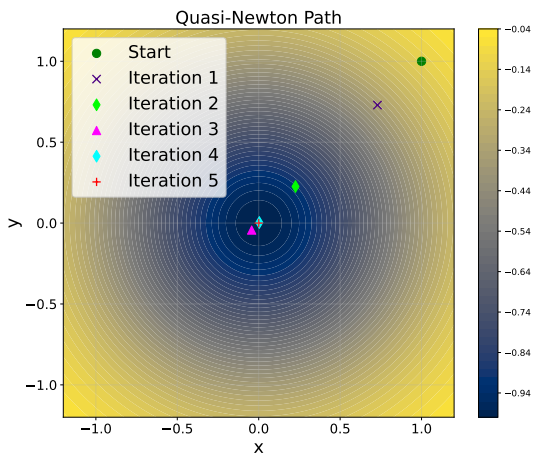
(d)

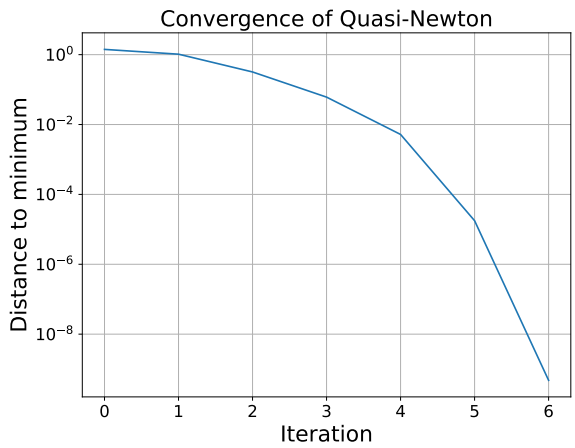
(e) The function proposed in the exercise has a very flat behaviour except for a tiny region around the minimum, meaning that the gradient is close to zero almost everywhere. The

Quasi-Newton method relies heavily on the gradient, which makes it inefficient for slowly varying functions.
There are a few options which may help in this case. One is to change the function to some other function with minima at the same points, but varies more rapidly, e.g.

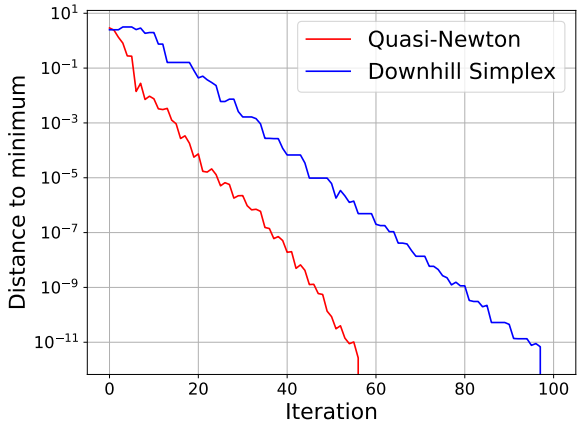
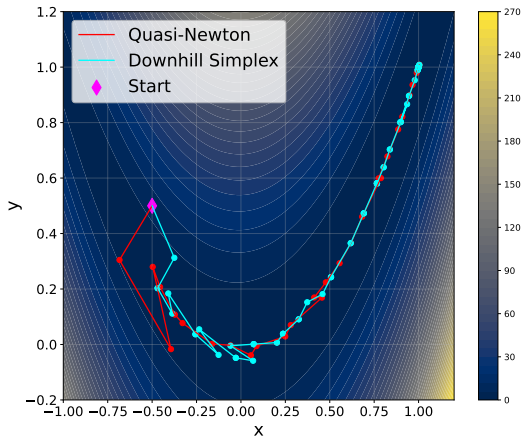
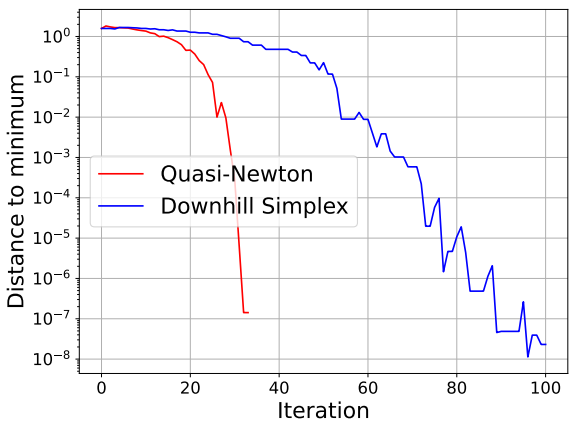
$$f(x, y) = -\exp\left(-x^2 - y^2\right) \quad \Rightarrow \quad f_{new}(x, y) = x^2 + y^2.$$

Another option is to rescale the function or start closer to the minimum.
In addition, don't forget to check not only the gradient, but also the step size for convergence.
To find optimal λ_i, check Wolfe conditions. Stop when, e.g. $\lambda_i \leq 10^{-8}$.
In addition, you can put a condition of resetting Hessian to a unity matrix if $\mathbf{\Delta}_i \cdot \mathbf{d}_i$ or $\mathbf{d}_i \cdot H_i \mathbf{d}_i$ is too small.
Figures below show the result for the original function $f(x, y)$ with the starting point $[1.0, 1.0]$.
The gradient points directly to the minimum, and the desired accuracy of 1e-8 is reached within 6 iterations.





(f) This function has a single minimum in $[0.0, 0.0]$. The plot below shows a comparison of convergence for the two methods for the starting point $[1.0, 1.0]$.



(g) The function $h(x, y)$ is called the Rosenbrock function and is widely used for testing optimisation algorithms. It has a single minimum at $[1.0, 1.0]$, localised in a steep parabolic valley. Hessian of this function has eigenvalues that differ in ~ 4 orders of magnitude (check yourself!). As a result, convergence to the minimum requires many steps in orthogonal directions (zig-zag).